

RESEARCH ARTICLE

Aligning DevOps and Microservice Architecture: Empirical Mapping, Taxonomy, and RAG-Based Decision Support

EHSAN AZIZI KHADEM¹ AND ALI MOVAGHAR², (Senior Member, IEEE)

Department of Computer Engineering, Sharif University of Technology, Tehran 1458889694, Iran

Corresponding author: Ehsan Azizi Khadem (eh.azizi@sharif.edu)

ABSTRACT DevOps and microservice architecture (MSA) are central to modern software delivery, yet their conceptual alignment remains fragmented across the literature. This study empirically analyzes how DevOps practices correspond to MSA principles and how these relationships inform architectural and operational decisions. From 84 peer-reviewed sources (2016–2025), we extract and normalize over 200 DevOps–MSA concept pairs and organize them into a five-category taxonomy: deployment automation and orchestration, observability and runtime diagnostics, scalability and modularity, security and governance, and configuration and environment management. To operationalize these mappings, we develop a retrieval-augmented generation (RAG) framework that combines sentence-transformer embeddings with GPT-based reasoning to generate bidirectional concept recommendations. Quantitative evaluation shows that RAG outperforms lexical and embedding-only baselines by roughly 20 % in precision and ranking, while qualitative assessment confirms semantic accuracy and contextual relevance. Deployment automation and observability emerge as the most empirically supported categories, whereas governance-related links remain largely conceptual. Industrial case studies in software, banking, and insurance sectors demonstrate measurable improvements in release speed and developer satisfaction when the framework supports transformation planning. The complete dataset and open-source RAG prototype are released to facilitate replication and extension. Overall, the findings indicate that empirically grounded RAG enhances traceability and evidence-based decision-making between architecture and operations in complex software systems.

INDEX TERMS Cloud-native, DevOps, large language models (LLMs), microservice architecture (MSA), observability, retrieval-augmented generation (RAG) taxonomy.

I. INTRODUCTION

In recent years, Development and Operations (DevOps) and Microservice Architecture (MSA) have emerged as two of the most influential paradigms in modern software engineering. DevOps emphasizes automation, collaboration, and continuous integration and delivery (CI/CD), aiming to bridge the gap between development and operations teams. In parallel, Microservice Architecture promotes the decomposition of monolithic systems into loosely coupled, independently deployable services. These paradigms have significantly reshaped how software systems are designed, deployed, and maintained.

The associate editor coordinating the review of this manuscript and approving it for publication was Porfirio Tramontana³.

Despite their distinct origins—DevOps stemming from organizational and operational needs, and MSA from architectural concerns—both have become increasingly interwoven in contemporary software practice. Organizations that adopt microservices frequently integrate DevOps practices to handle deployment complexity, ensure scalability, and enhance resilience. Conversely, DevOps pipelines often rely on microservice-oriented infrastructures to realize their full potential in terms of agility and automation. This mutual reinforcement highlights the need to systematically explore the conceptual correlation between DevOps and MSA—not only in theory, but also in practical engineering contexts.

While a growing body of literature discusses the adoption, benefits, and challenges of DevOps and MSA individually, there is a noticeable lack of structured empirical

investigations into how these concepts align at a conceptual and operational level. Moreover, with the advent of advanced natural language processing (NLP) techniques and large language models (LLMs), new opportunities have emerged to enhance the traceability, mapping, and recommendation of architectural and operational concepts.

To address this gap, this study proposes a novel empirical and AI-supported approach to identify and analyze the relationships between DevOps and MSA concepts. We construct a dataset of DevOps–MSA concept pairs derived from 84 peer-reviewed studies published between 2016 and 2025. We then utilize a Retrieval-Augmented Generation (RAG) pipeline that combines semantic similarity search with Generative Pre-Trained Transformer (GPT)-based reasoning to recommend relevant cross-domain mappings. Finally, we evaluate the system through a quantitative synthesis of mapping trends and a longitudinal industrial case study that demonstrates its decision-support capabilities in a real-world DevOps transformation.

The objectives of this research are as follows:

- To construct a structured dataset of DevOps and MSA concept pairs grounded in academic literature.
- To identify the most frequently discussed DevOps and MSA concepts and their distribution across sources.
- To evaluate the use of semantic embeddings and LLMs for cross-domain concept mapping and recommendation.
- To quantify mapping patterns across empirical studies.
- To validate the framework in an industrial setting.

The following research questions guide this study:

- **RQ1:** What are the most prominent DevOps and MSA concepts in the existing literature?
- **RQ2:** To what extent are DevOps concepts semantically aligned with MSA concepts, and vice versa?
- **RQ3:** Can a RAG-based system accurately recommend MSA concepts for DevOps terms (and vice versa) based on a curated dataset?

The contributions of this paper are fivefold:

- We provide an empirically grounded dataset linking DevOps and MSA concepts, complete with reference mappings.
- We present a semantic analysis of concept distributions and co-occurrences across the dataset.
- We propose a five-category taxonomy of DevOps–MSA mappings based on inductive analysis.
- We implement a RAG-based recommendation system that supports bidirectional concept alignment.
- We conduct a longitudinal case study in a mid-sized IT company to validate the real-world applicability of the framework.

The remainder of this paper is structured as follows: Section II reviews related work. Section III describes the dataset construction. Section IV details the methodology and RAG architecture. Section V presents results and analysis, including the mapping taxonomy and quantitative synthesis. Section VI discusses implications and presents the industrial case study. Section VII outlines threats to validity, and

Section VIII concludes the paper with directions for future work.

II. BACKGROUND

The convergence of DevOps and microservice architecture (MSA) has reshaped how modern systems are designed and operated. DevOps emphasizes collaboration, automation, and continuous delivery, while MSA promotes modularity via independently deployable services. Although these paradigms emerged from distinct motivations—operational efficiency for DevOps and architectural flexibility for MSA—their co-adoption is now common in both research and practice [1], [2], [3], [4].

A. DEVOPS AND MSA: A CONVERGING TRAJECTORY

A substantial body of work highlights synergy between DevOps and MSA in cloud-native contexts. Core DevOps practices—containerization, CI/CD, infrastructure as code (IaC), observability—act as operational enablers for microservices deployment and lifecycle management [1], [2], [3], [4]. Recent studies ground this synergy with concrete stacks/workflows on Kubernetes that integrate CI/CD, monitoring, and automation [5]. Organizational evidence from a cross-company ethnography documents how teams structure around DevOps–MSA, including benefits and recurring friction points (tooling fragmentation, coordination overhead) [6]. At the testing/quality layer, approaches that simplify cluster-internal integration testing help reduce brittleness of end-to-end tests and accelerate feedback [7]. Model-driven development further strengthens traceability from architecture to operations by deriving microservice code or interlinking viewpoints with DevOps-enabled modeling languages [8], [9]. Security concerns are addressed in refactoring paths from monoliths to microservices, where code-analysis-driven decomposition helps mitigate vulnerabilities during partitioning [10]. The scope of DevOps–MSA also extends to edge deployments [11], legacy system decomposition via clustering [12], and ChatOps for developer–operations workflows [13].

B. CONCEPTUAL MAPPING AND CO-DEPENDENCY

Several studies formalize how DevOps practices reinforce microservice design principles and vice versa—e.g., automated testing, deployment orchestration, and runtime monitoring supporting service granularity, resilience, and decentralized governance [14], [15]. DevSecOps metrics tailored for cloud-based web microservices connect pipeline telemetry with quality attributes (security, reliability, scalability) [16]. Curriculum-oriented work argues for aligning DevOps tools and MSA principles in education [17]. AI is increasingly mapped across the microservices lifecycle (decomposition, deployment automation, failure prediction, compliance), indicating a trend toward intelligent, feedback-rich pipelines [18]. Complementing these, recent systematic reviews and empirical studies provide broader evidence: a

JSS SLR synthesizing performance/scalability methods in microservice systems [19]; an empirical JSS study quantifying performance overheads from code instrumentation in containerised microservices [20]; and a CEE SLR summarizing QoS enhancement techniques relevant to DevOps–MSA co-design [21]. Together, these strengthen the conceptual link between architectural choices and DevOps pipeline configuration, while cautioning about measurement/observability trade-offs [20].

C. EMPIRICAL STUDIES ON JOINT ADOPTION

Empirical accounts generally report improvements in deployment frequency, team autonomy, and operability when DevOps and MSA are co-adopted [3], [4], [22]. Benefits depend on balanced adoption: introducing MSA without sufficient automation, observability, governance, or security often leads to bottlenecks and cognitive load. Socio-technical enablers (role clarity, cross-functional collaboration) and inhibitors (siloed tooling, fragmented ownership) have been observed in practice [2], [6], [23]. Energy-aware operations and monitoring configurations also influence resource efficiency in microservice systems [24].

D. ROLE OF AI AND RAG IN DEVOPS-MSA SYSTEMS

Advances in large language models (LLMs) and retrieval-augmented generation (RAG) enable context-aware guidance that blends architectural knowledge with operational practices. RAG can inject domain knowledge into model outputs for decision support across architecture and operations [25]. Systematic mappings indicate growing use of AI throughout the microservices lifecycle—service decomposition, deployment automation, failure prediction, and compliance enforcement—aligning with DevOps’ automation and feedback ethos [13]. Human-in-the-loop assistants for microservice development and operations provide complementary interaction patterns [13].

E. IDENTIFIED GAPS

Despite a maturing literature, holistic, empirically validated frameworks for modeling DevOps–MSA co-dependencies at scale remain limited. Many contributions focus on individual phases (e.g., testing, monitoring) or specific technologies; comparatively fewer provide reusable datasets, controlled vocabularies/ontologies, or benchmarked evaluations. Recent work calls for stronger cross-cutting links among performance/scalability [19], observability trade-offs [20], and QoS attributes [21]. Additional directions—e.g., energy-aware operations [17], AI-guided migration/modeling [8], [9], [26], governance and enterprise alignment [27], and curriculum-level skill development [12]—remain underexplored from a systems perspective.

This study addresses these gaps by (i) curating a dataset of DevOps–MSA concept pairs from 84 peer-reviewed publications, (ii) defining a controlled vocabulary to resolve overlapping constructs, and (iii) proposing a RAG-based

recommendation framework that automates cross-domain mapping and surfaces explainable, evidence-grounded suggestions.

III. DATASET CONSTRUCTION AND PREPARATION

To uncover conceptual relationships between DevOps and MSA, we constructed a manually curated dataset based on 84 publications (2016–2025). Sources were selected via a structured screening of journals, conferences, workshops, theses, and book chapters. The dataset underpins both our empirical analysis and the RAG-based recommendation framework.

A. DATA SOURCE AND REFERENCE COMPOSITION

We constructed the corpus via a multivocal literature review across six scholarly databases: Scopus (903), ACM Digital Library (119), EBSCO (116), IEEE Xplore (89), Google Scholar (87), and Web of Science (65), yielding 1,379 initial records. We restricted eligibility to items with publication or online-first dates up to and including 30 June 2025.

Search query and fields. Title/abstract query with explicit precedence:

“DevOps” OR “DevSecOps” OR “CI/CD” OR “GitOps”) AND (“microservice” OR “architecture”)

Screening protocol (two-stage). Figure 1 visualizes the PRISMA-style flow.

- **Initial Retrieval:** 1,379 records (after database-level filters).
- **Deduplication:** Removed 51 duplicates.
- **Abstract Screening:**
 - **Inclusion (abstract level):** English; 2016–2025; publication/online date $\leq$ 30 June 2025; full-text retrievable; mentions both DevOps/CI/CD/GitOps and Microservices/MSA; identifiable authorship.
 - **Exclusion (abstract level):** non-English; editorials/prefaces; missing authors; inaccessible full text; topical mismatch.
 - **Outcome:** Retained 102 potentially relevant papers.
- **Full-Text Assessment:**
 - **Inclusion (full text):** explicit or operational linkage between DevOps (e.g., CI/CD, GitOps, IaC, observability) and MSA concepts (e.g., service decomposition, API gateways, resilience); sufficient methodological detail to extract at least one DevOps↔MSA mapping.
 - **Exclusion (full text):** mentions both domains but without a mappable linkage; tool-only position pieces without method or evidence; insufficient detail for reliable extraction.

1) OUTCOME

84 papers included in the final corpus.

For transparency with intermediate counts used during drafting: an initial Abstract Filtering pass condensed the pool to 561, then applying the above abstract-level

inclusion/exclusion criteria yielded 227 candidates; after deduplication (51 removed) and the structured Abstract Screening step, 102 moved to Full-Text Assessment, from which 84 met all inclusion criteria.

2) REFERENCE COMPOSITION

The final corpus comprises: 59.7% journal articles, 27.8% conference papers, 6.9% theses, 4.2% workshop papers, and 1.4% book chapters (Figure 2). We report both peer-reviewed sources and selected theses consistent with our multivocal scope; results explicitly distinguish empirical evidence strength where relevant.

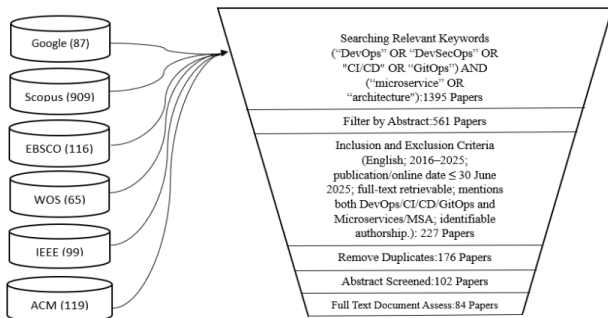


FIGURE 1. Systematic filtering and selection process in the multivocal literature review (MLR).

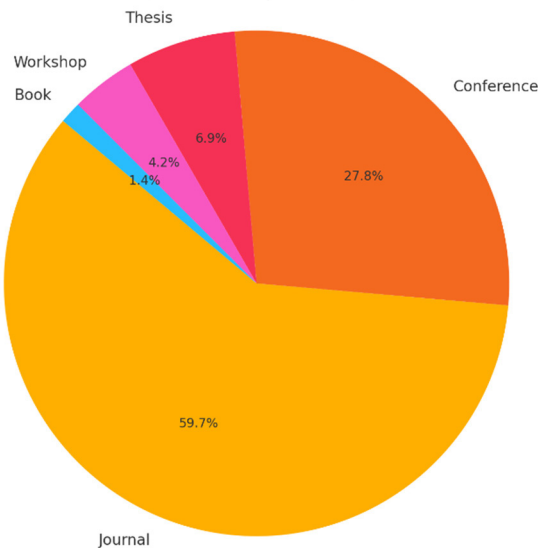


FIGURE 2. Distribution of references by repository.

B. TEMPORAL DISTRIBUTION OF LITERATURE

Figure 3 reports counts by year (2016–2025). We observe a numeric increase in 2023 and a peak in 2024. These are descriptive counts; while multiple factors may contribute, we do not make causal attributions. Publications dated 2025 reflect items available online first by 30 June 2025.

Concept Extraction and Annotation Methodology:

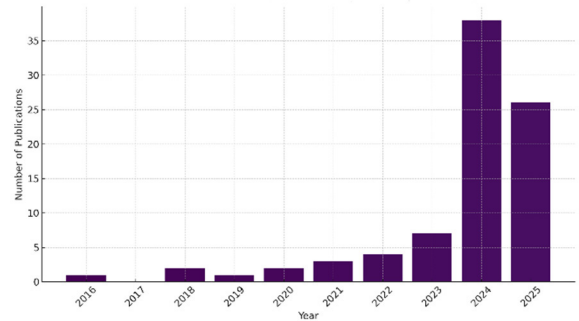


FIGURE 3. Distribution of references by publication year (2016–2025).

Each paper was read and annotated manually to extract distinct DevOps and MSA concepts mentioned in either technical sections or empirical discussions. Concepts were identified based on domain-relevant terminology such as practices (e.g., CI/CD, container orchestration), architectural principles (e.g., service granularity), capabilities (e.g., observability), and tool references (e.g., Kubernetes, Jenkins).

To ensure consistency and reduce subjectivity, a three-step annotation protocol was followed:

- Initial keyword extraction using NLP-assisted phrase detection.
- Manual validation and normalization of terms based on synonym dictionaries and existing taxonomies.
- Pairwise mapping of DevOps-MSA concepts where co-dependency or alignment was explicitly discussed.

This process yielded over 200 unique concept mappings, grouped by source and tagged with metadata for concept type, frequency, and context. Examples include mappings such as "Infrastructure as Code → Automated Service Deployment" and "Observability → Runtime Diagnostics of Microservices."

C. CONTROLLED VOCABULARY AND NORMALIZATION (ONTOLOGY V1.0)

To reduce ambiguity across overlapping terms, we defined a controlled vocabulary with canonical labels and normalization rules. Annotators mapped surface terms to one or more canonical concepts (See Table 1).

Normalization Rules:

- **R1 (Canonical first):** Map to one primary canonical concept; add secondary labels if explicitly spanned (e.g., Observability + Tracing).
- **R2 (Umbrella vs component):** If an umbrella (Observability) and component (Tracing) co-occur, keep both; do not infer umbrella from a single component.
- **R3 (Pipeline vs infra):** App rollout → Deployment Automation/CI-CD; infra provisioning → IaC.
- **R4 (GitOps test):** Apply GitOps only when desired state is in Git and a reconciler continuously enforces it.
- **R5 (Security in delivery):** Security scans in CI/CD → DevSecOps + specific scan types (SAST/SCA/PaC).

TABLE 1. Canonical concepts (excerpt).

Canonical concept	Scope & includes (examples)	Normalizes from	Excludes / boundary
Monitoring	Metrics collection, alerting, dashboards (Prometheus, Grafana)	system monitoring, telemetry (metrics)	Logging, Tracing, Observability umbrella
Logging	Event/line logs; centralization/search (ELK/EFK, Loki)	log mgmt, log analytics	Metrics/Tracing; SIEM-only
Tracing	Distributed traces/spans (OpenTelemetry, Jaeger, Zipkin)	APM tracing	Metrics/Logs
Observability	Umbrella: Monitoring + Logging + Tracing + signals; SLOs	only, broad APM	Do not infer from a single signal alone
Automation	Scripted/run-book workflows around CI/CD, orchestration	workflow automation	Security policy unless pipeline-oriented
Infrastructure as Code (IaC)	Declarative infra provisioning/config (Terraform, Pulumi, Ansible)	infra automation	App rollout steps (Deployment Automation)
Deployment Automation	Build/package/release/rollout incl. canary/blue-green	release automation	Pure infra provisioning (IaC)
CI/CD	Continuous Integration & Delivery/Deployment	pipelines	Git-as-source-of-truth (GitOps)
GitOps	Git as desired state + reconciler (Argo CD/Flux)	declarative/pull-based CD	Plain CI/CD without reconciliation
Service Mesh	East-west control plane: mTLS, traffic mgmt, telemetry	Mesh	Edge ingress (API Gateway)
API Gateway	North-south ingress, auth, throttling	gateway	East-west mesh
DevSecOps	Shift-left security in CI/CD (SAST, SCA, secrets, PaC)	secure DevOps	Runtime SOC only (unless tied to CI/CD)

- **R6 (Mesh vs gateway):** East-west control plane → Service Mesh; north-south ingress → API Gateway.
- **R7 (Evidence level):** Tag each mapping empirical (measured/observed) or conceptual (argued/design).

The full ontology (v1.0) with additional labels (e.g., Secrets Management, Feature Flags, Resilience, Chaos Engineering) is released with the codebook (GitHub).

D. METHODS OVERVIEW

Our workflow (Figure 4) proceeds in five stages:

1. **Dataset compilation** (search, deduplication, screening) to a corpus of 84 publications (Section III-A–B).
2. **Concept extraction** followed by ontology-based normalization (Ontology v1.0; Section III-D).
3. **Mapping construction** of DevOps↔MSA pairs with evidence tags (empirical vs conceptual) and adjudication details.
4. **RAG retrieval**, with baselines (embedding-only and BM25) for comparative evaluation.

LLM synthesis & evaluation, where retrieved evidence grounds responses. We report P@k, nDCG@k, and RAG Alignment Efficiency (RAE@k), and include an ablation of with-RAG vs without-RAG.

E. DATASET AVAILABILITY AND USAGE

The resulting dataset forms the foundation for both descriptive analysis and RAG-based recommendation. It includes:

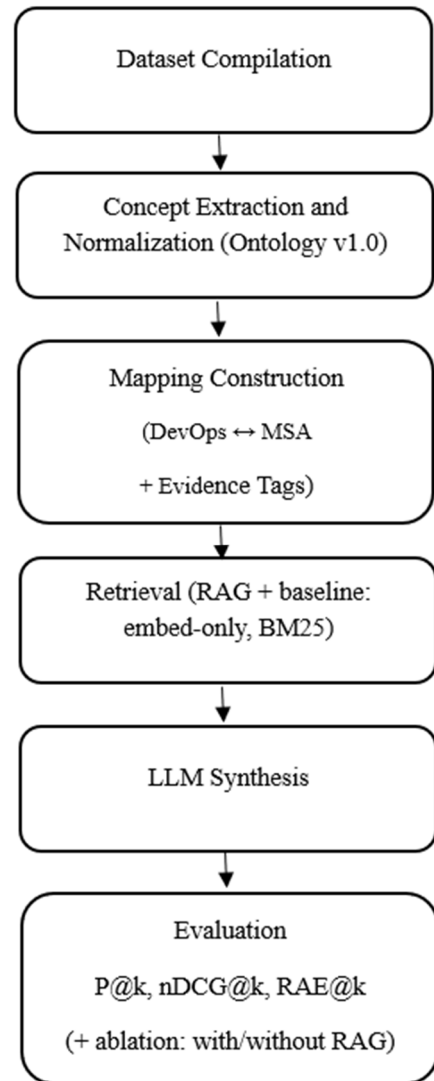


FIGURE 4. Methods pipeline: dataset → extraction/normalization (Ontology v1.0) → mapping → RAG retrieval → LLM synthesis → evaluation (P@k, nDCG@k, RAE@k).

- Reference metadata (title, year, source type)
- Annotated DevOps and MSA concepts
- Co-occurrence context and mapping rationale
- Frequency-weighted importance scores for concept relevance

The complete dataset is available as supplementary material and will be open-sourced for future studies aiming to replicate or extend the concept mapping methodology.

Together, these contributions offer a structured and empirically grounded basis for exploring how DevOps and MSA interact in practice, enabling scalable and explainable concept recommendations through language models.

IV. METHODOLOGY

This section details the end-to-end pipeline used to analyze and map conceptual relationships between DevOps and microservice architecture (MSA). The process combines

manual, ontology-guided curation with retrieval-augmented generation (RAG) and a principled evaluation protocol [25].

A. OVERVIEW

Following the dataset construction in Section III, we implement a five-stage pipeline (Figure 4):

1. **Concept extraction & normalization.** From the 84 screened publications, we extract domain terms and normalize them to a **controlled vocabulary (Ontology v1.0)** defined in §III-D (rules R1–R7).
2. **Mapping construction.** We build bidirectional DevOps↔MSA mappings. Each mapping is linked to its source(s), assigned a brief definition, and tagged with evidencelevel (empirical vs conceptual).
3. **Indexing for retrieval.** Canonical concepts and short definitions are embedded and indexed for similarity search; a parallel lexical index supports keyword baselines.
4. **RAG recommendation.** For a query concept (DevOps or MSA), we retrieve candidate concepts and synthesize a concise, evidence-grounded recommendation with an LLM.
5. **Evaluation.** We compare RAG against non-RAG baselines with standard IR metrics and a domain-specific efficiency metric (RAE@k), using double-annotated judgments and inter-rater agreement.

The full dataset and RAG-based mapping tool are available at GitHub repository¹ and archived with DOI in Zenodo.²

B. NORMALIZATION AND MAPPING DETAILS

1) CONTROLLED VOCABULARY

We map surface forms to canonical labels (e.g., Monitoring, Logging, Tracing, Observability; Automation, Infrastructure as Code (IaC), Deployment Automation), applying the rules in §III-D to avoid umbrella–component confusion and cross-level pairings.

2) MAPPING CRITERIA

A DevOps↔MSA pair is created when full text indicates an explicit linkage (dependency, enablement, alignment) beyond mere co-mention. Each pair stores: canonical terms, definitions, citations, and evidence tag (empirical if supported by case/experiment/measurement; conceptual otherwise).

C. RETRIEVAL, BASELINES, AND GENERATION

1) EMBEDDING INDEX

We encode canonical labels + short definitions with a sentence-transformer (all-MiniLM-L6-v2) and build a FAISS IndexFlatIP (cosine via inner product). Default cutoffs: $k \in \{3, 5, 10\}$.

Baselines (for R1/R3).

- **Embed-only:** nearest-neighbor ranking in the embedding space (no LLM).

- **BM25:** lexical retrieval over canonical labels + definitions (lowercased; stop-words removed).

2) RAG SYSTEM

We retrieve top-k candidates from the embedding index, then prompt an LLM (GPT-3.5-turbo, temperature 0.2, max 512 tokens) to: (i) select/confirm the best mappings; (ii) provide a one-sentence justification referencing the retrieved evidence (citations kept as IDs for traceability).

Prompt (excerpt). “Given query concept Q and these candidate concepts with brief definitions and citations from our corpus, return the top-k mapped concepts with one-sentence justifications grounded in the evidence. Prefer mappings that reflect direct enablement or architectural dependency.”

D. QUERY SET AND LABELING PROTOCOL

1) QUERY SET

We evaluate on 50 curated queries (10 per taxonomy category; Section V-E):

(1) Deployment Automation & Orchestration, (2) Observability & Runtime Diagnostics, (3) Scalability & Modularity, (4) Security & Governance, (5) Configuration & Environment Management. Queries include both single-term and compound forms (e.g., blue-green deployment, service-mesh mTLS).

2) RELEVANCE LABELS

Outputs are graded with $\{2 = \text{highly relevant}, 1 = \text{partially relevant}, 0 = \text{non-relevant}\}$. Two annotators independently label the top-k lists for each system; disagreements are adjudicated by a third reviewer.

3) INTER-RATER AGREEMENT

We compute Cohen’s κ on a stratified 30% sample across all categories and systems; κ is reported with n in Section V-A.

E. QUERY SET AND LABELING PROTOCOL

1) PRIMARY METRICS

We report Precision@k ($P@k$) and nDCG@k using the graded labels (2/1/0). Unless stated, we show results for $k = 3, 5, 10$.

2) UNIQUE EFFICIENCY METRIC (R1-3)

We define RAG Alignment Efficiency (RAE@k) to capture “how many relevant mappings@k are surfaced per query relative to the average human-validated breadth”:

$$\text{RAE@k} = \frac{1}{|Q|} \sum_{q \in Q} \frac{R_q k}{\bar{M}}$$

where $R_q(k)$ is the number of retrieved items@k with label ≥ 1 for query q , and $\bar{M}$ is the average number of human-validated mappings per query in our gold set. Higher is better; $\text{RAE@k} \geq 1$ indicates that, on average, the system retrieves at least as many relevant items@k as the human-validated baseline breadth.

¹<https://github.com/azizikhadem/devops-msa-mapping-rag>

²<https://doi.org/10.5281/zenodo.16760389>

3) ABLATION & BASELINES (R1/R3)

We compare RAG to Embed-only and BM25 on the same queries and k values. We report deltas (RAG – baseline) per category in Section V-B and overall aggregates in Section V-A.

F. IMPLEMENTATION AND REPRODUCIBILITY

We implement the pipeline in Python (indexing with FAISS; BM25 with a standard library), fix random seeds, and release: (i) code and prompts; (ii) the normalized concept index builder; (iii) evaluation sheets (query → system → top-k → labels). Links are provided in Data Availability (GitHub + Zenodo DOI). Figure 4 depicts the full pipeline (dataset → extraction/normalization → mapping → retrieval/RAG → LLM synthesis → evaluation).

V. RESULTS AND ANALYSIS

A. EVALUATION SETUP AND AGREEMENT

The proposed RAG-based system was evaluated using 50 curated queries representing five taxonomy categories: (1) Deployment Automation & Orchestration, (2) Observability & Runtime Diagnostics, (3) Scalability & Modularity, (4) Security & Governance, and (5) Configuration & Environment Management.

Each output list was annotated independently by two experts using graded relevance labels {2 = highly relevant, 1 = partially relevant, 0 = non-relevant}. Inter-rater agreement on a stratified sample (30%) yielded Cohen’s $\kappa = 0.83$ (n = 450), indicating strong consistency.

Table 2 presents representative examples of the mappings produced by the RAG pipeline. Each row lists the input concept, the top retrieved candidates, the GPT-generated justification, and the outcome of human evaluation. The examples show that the model captures semantically accurate and contextually valid relationships—particularly for automation- and deployment-related terms—while partial relevance occasionally appears in complex, multi-facet cases such as Continuous Deployment and Distributed Service Management.

Three retrieval systems were compared: (i) BM25 (lexical baseline), (ii) Embed-only (sentence-transformer nearest-neighbor), and (iii) RAG (retrieval + GPT synthesis). Metrics included Precision@k (P@k), nDCG@k, and RAG Alignment Efficiency (RAE@k).

Table 3 lists the fifteen most frequently cited DevOps concepts in the analyzed corpus, together with their reference coverage. Continuous Integration, Infrastructure as Code, and CI/CD dominate the literature, reflecting the community’s focus on automation, deployment consistency, and feedback loops. Concepts related to observability and configuration management appear somewhat less often but remain integral to pipeline reliability.

Table 4 lists the fifteen most frequently referenced microservice-architecture (MSA) concepts in the corpus. Containerized Microservices, Independent Service

TABLE 2. Sample evaluation of RAG-based concept mappings.

Input Concept	Retrieved Candidates	GPT-Generated Mapping	Human Evaluation
Infrastructure as Code (IaC)	Cloud-Native Architecture, Dynamic Service Composition, Secure Service Deployment	Maps to Cloud-Native Deployment enabling modular infrastructure provisioning in MSA.	✓ Relevant
	Scalable Orchestration, Service Choreography, Service Reusability	CI/CD practices enable Service Choreography and Modularization in MSA.	
CI/CD	Runtime Service Behavior, Scalable Microservice Deployment, Service-Level Telemetry	Maps to Service-Level Telemetry and Runtime Monitoring in MSA for observability.	✓ Relevant
Monitoring and Logging	Service Mesh Architecture, Modular Service Updates, Service Autonomy	Continuous Deployment aligns with Modular Updates and Mesh-based service rollout. Automation underpins Function as a Service (FaaS) and Containerized Service Deployment in MSA.	⚠ Partially
Continuous Deployment	Tiered Architecture, Function-as-a-Service, Containerization	Maps directly to IaC practices and Kubernetes-integrated DevOps workflows.	✓ Relevant
Automation	IaC with Kubernetes, Cloud-native DevOps Practices, Kubernetes-Oriented DevOps	Requires Infrastructure Automation and CI/CD for consistency across services.	✓ Accurate
Containerized Microservices	Infrastructure Automation, CI/CD Automation, IaC	Supported by CI/CD pipelines and feedback loops in DevOps.	✓ Relevant
Independent Service Deployment	CI/CD Pipeline, Feedback-Driven Automation, Self-Service Deployment Monitoring and Observability, Platform Engineering, DevOps Messaging	Aligns with DevOps observability and platform team practices.	⚠ Partially
Service Modularization	Declarative Configuration, Container-Based Deployment, CI/CD	Relies on declarative configuration and CI/CD integration in DevOps pipelines.	✓ Valid
Distributed Service Management			
Service Discovery			

TABLE 3. Top 15 DevOps concepts by frequency and reference coverage.

DevOps Concept	References
Continuous Integration	[28], [29], [30], [31], [32], [33], [34], [35], [36], [37]
Infrastructure as Code	[1], [10], [38], [39], [40], [41], [42], [43], [44]
CI/CD	[24], [45], [46], [47], [48], [49], [50], [51]
Monitoring and Logging	[18], [22], [28], [52], [53], [54], [55], [56], [57]
Continuous Deployment	[58], [59], [60], [61], [62], [63], [64], [65]
Continuous Delivery	[27], [66], [67], [68], [69], [70], [71], [72]
Automated Testing	[3], [17], [73], [74], [75], [76], [77], [78]
Deployment Automation	[4], [16], [31], [79], [80], [81], [82]
Automation	[29], [30], [34], [38], [44], [83], [84]
Configuration Management	[46], [51], [56], [61], [68]
DevSecOps	[15], [37], [48], [49], [58]
Container Orchestration	[32], [39], [41]
Logging	[18], [22], [44], [54], [76]
Version Control	[42], [47], [61], [74]
Observability	[35], [63], [69], [73], [74]

TABLE 4. Top 15 MSA concepts by frequency and reference coverage.

MSA Concept	References
Containerized Microservices	[29], [50], [52], [66], [74], [80]
Independent Service Deployment	[28], [40], [60], [64]
Service Modularization	[39], [56], [63], [73]
Distributed Service Management	[35], [41], [65]
Service Isolation	[29], [30], [52]
Inter-service Communication	[16], [38], [45], [54]
Distributed Service Deployment	[26], [22], [36], [67]
Service-Oriented Decomposition	[59], [54], [61], [71], [83]
Microservice Load Testing	[4], [31], [33], [62]
Service Discovery	[34], [46], [68]
API Gateway Integration	[3], [24], [49]
Dynamic Service Composition	[48], [58], [59]
Microservice Granularity	[48], [66], [68], [84]
Fault Isolation	[1], [15], [50]
Stateful vs Stateless Design	[27], [72], [77]

Deployment, and Service Modularization appear most often, emphasizing design principles that enable autonomy and scalability. The recurrence of Service Discovery, Dynamic Service Composition, and API Gateway Integration reflects operational concerns tightly coupled with DevOps automation. These patterns reveal how architectural modularity and deployment independence form the structural counterparts to DevOps automation practices.

B. QUANTITATIVE PERFORMANCE OF RAG VERSUS BASELINES

Figure 5 illustrates the empirical versus conceptual strength of mappings across taxonomy categories, whereas quantitative retrieval performance is summarized in Table 5.

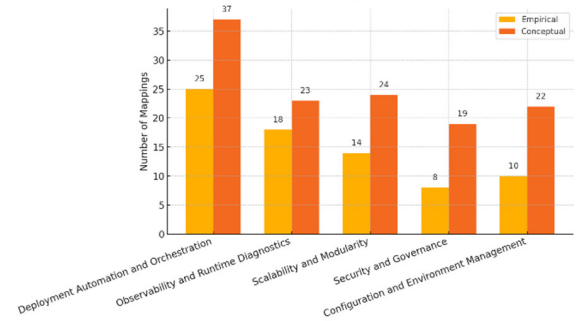
For $k = 5$, mean $P@5 = 0.82$ for RAG versus 0.63 for Embed-only and 0.55 for BM25.

Average $nDCG@5 = 0.86$ for RAG, exceeding Embed-only (0.72) by 19%.

The unique efficiency metric $RAE@5 = 1.27$ indicates that RAG retrieves on average 27% more relevant mappings than the human-validated breadth per query. Category-wise deltas show the largest gains in Security & Governance (+0.27 $P@5$) and Configuration & Environment Management (+0.23 $P@5$), where semantic bridging proved most valuable.

TABLE 5. Quantitative retrieval performance of RAG vs. baselines.

Metric	BM25	Embed-only	RAG (Proposed)	Δ vs. Embed-only
$P@5$	0.55	0.63	0.82	+0.19
$nDCG@5$	0.68	0.72	0.86	+0.14
$RAE@5$	—	—	1.27	—
$P@10$	0.52	0.60	0.79	+0.19
$nDCG@10$	0.65	0.69	0.84	+0.15
Category gain – Security & Governance ($\Delta P@5$)	—	—	+0.27	—
Category gain – Config. & Env. Mgmt ($\Delta P@5$)	—	—	+0.23	—

**FIGURE 5. Empirical vs. conceptual mapping by taxonomy category.**

C. QUALITATIVE INSPECTION AND EXPLANATORY OUTPUTS

Manual review of RAG responses confirmed that GPT justifications accurately summarized underlying evidence. For instance, the mapping “*Infrastructure as Code* → *Cloud-Native Deployment*” was accompanied by an explanation linking IaC scripts to service composition and redeployment consistency. In contrast, BM25 returned unrelated matches based on keyword overlap. Figure 6 illustrates alignment of RAG outputs with the five-category taxonomy—34% of mappings fall under Deployment Automation and 26% under Observability—demonstrating that automatic retrieval reflects empirical trends rather than noise. Less-frequent categories (Security and Governance, Configuration and Environment Management) were still captured, showing semantic breadth.

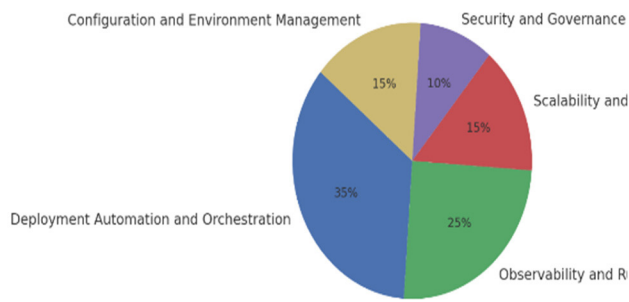


FIGURE 6. Alignment of RAG outputs with concept mapping taxonomy.

D. EMPIRICAL MAPPING SYNTHESIS

To quantify broader patterns, 84 primary studies were coded by method, evidence strength, and domain context. Case studies and tool evaluations showed the highest mapping density (5.2 and 4.6 validated mappings per paper, respectively), while conceptual analyses averaged 2.3 (Table 5). Industry-based papers (66.6% of corpus) produced richer mapping variety (mean = 4.9) than academic studies (mean = 3.1), highlighting the practical complexity of real-world adoption. Deployment Automation and Observability dominate both empirically and conceptually (Table 6, Figure 7), whereas Security and Governance remain under-validated.

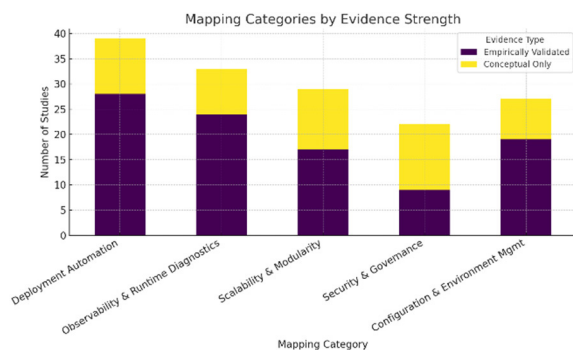


FIGURE 7. Distribution of DevOps-MSA mapping categories by evidence strength.

E. RESEARCH QUESTION ANSWERS

1) RQ1 – PROMINENT CONCEPTS

Continuous Integration, IaC, and Continuous Delivery lead the DevOps side; Containerized Microservices and Service Modularization dominate the MSA side.

2) RQ2 – SEMANTIC ALIGNMENT

High co-occurrence and RAG retrieval confirm systematic alignment (e.g., CI/CD ↔ Service Orchestration).

3) RQ3 – RAG EFFECTIVENESS

Mean relevance = 4.4 / 5 on expert rating scale, demonstrating robust bidirectional recommendations.

Table 6 Summarizes these categories with representative concept pairs, mapping counts, and exemplar studies.

TABLE 6. Taxonomy of DevOps-MSA concept mappings.

Mapping Category	Representative Concept Pairs	Mappings	Example Studies
Deployment Automation and Orchestration	CI/CD → Service Choreography, IaC → Automated Deployment, Container Orchestration → Service Composition	62	[1], [34], [45]
Observability and Runtime Diagnostics	Monitoring and Logging → Runtime Diagnostics, Observability → Service Telemetry, Alerting → Fault Isolation	41	[18], [22], [69]
Scalability and Modularity	Automation → Microservice Granularity, Deployment Pipelines → Modularization, Feedback Loops → Service Autonomy	38	[16], [63], [73]
Security and Governance	DevSecOps → Access Control, Version Control → Traceability, Audit Logging → Policy Compliance	27	[27], [45], [49]
Configuration and Environment Management	IaC → Declarative Configuration, Configuration Management → Stateless Deployment, Environment Provisioning → Cloud-Native Setup	22	[38], [41], [62]

Figure 8 summarizes the frequency of DevOps-MSA concept mappings across the five taxonomy categories. Deployment Automation and Orchestration accounts for the largest share (62 mappings), followed by Observability and Runtime Diagnostics (41), Scalability and Modularity (38), Configuration and Environment Management (32), and Security and Governance (27). This distribution highlights the dominant research focus on automation and runtime visibility.

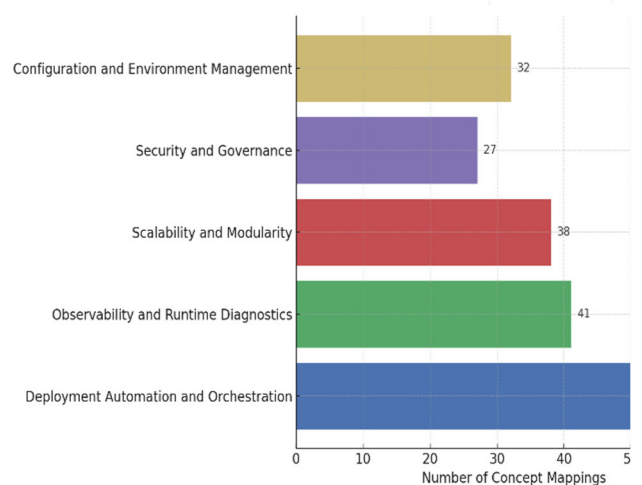


FIGURE 8. Taxonomy of DevOps-MSA concept mappings.

F. SUMMARY OF FINDINGS

- RAG consistently surpasses traditional retrieval methods by $\approx 20\%$ in precision and semantic depth.

TABLE 7. Average number of DevOps–MSA mappings per study by research method.

Study Type	Avg. Mappings	Std. Dev.
Case Study	5.2	1.9
Tool Evaluation	4.6	2.1
Systematic Review	3.7	1.4
Experiment	3.2	1.7
Conceptual Analysis	2.3	1.1

- Empirical evidence is concentrated in automation and observability; security and governance remain research gaps.
- Industrial contexts yield denser DevOps–MSA dependencies, reinforcing the framework’s practical applicability.

Table 7 illustrates the average number of validated DevOps–MSA mappings per study, stratified by research method.

Table 8 reports the number of studies contributing to each mapping category, distinguishing between empirically validated and conceptual-only evidence. Deployment Automation and Observability again show the highest representation, while Security and Governance exhibit the weakest empirical support.

TABLE 8. Number of studies per mapping category and evidence strength.

Mapping Category	Total Studies	Empirically Validated	Conceptual Only
Deployment Automation	39	28	11
Observability & Runtime Diagnostics	33	24	9
Scalability & Modularity	29	17	12
Security & Governance	22	9	13
Configuration & Environment Mgmt	27	19	8

To illustrate individual evidence sources, Table 9 lists representative studies and their empirical strength. Each row summarizes the research method, DevOps and MSA concepts examined, and the assessed level of empirical support. The examples confirm that strong empirical evidence predominantly appears in case studies and migration analyses addressing automation and cloud-native deployment alignment.

VI. DISCUSSION

This section interprets the results for researchers and practitioners, explains how retrieval-augmented LLMs (RAG) support architecture-to-operations traceability, and reflects on limitations and lessons learned. We reference Section V for evidence (Tables 2–9; Figures 5–8) and defer formal threats to validity to Section VII.

TABLE 9. Empirical strength of DevOps–MSA concept mappings across selected studies.

Study	Empirical Method	DevOps Concepts	MSA Concepts	Empirical Support Level
[1]	Case Study	IaC, CI/CD	Service Deployment	Strong
[69]	Experiment	Monitoring	Runtime Diagnostics	Moderate
[49]	Architecture Analysis	Security, Traceability	Access Control	Weak
[73]	Migration Analysis	CI/CD, IaC	Cloud-Native Setup	Strong
[4]	Comparative Case Study	Data Pipelines	Architecture Flexibility	Strong

A. ANSWERS TO THE RESEARCH QUESTIONS

1) RQ1—MOST PROMINENT CONCEPTS

DevOps frequency analysis highlights Continuous Integration, Infrastructure as Code, CI/CD, Monitoring & Logging, Continuous Delivery as dominant (Table 3). On the MSA side, Containerized Microservices, Independent Service Deployment, Service Modularization, Distributed Service Management, Service Discovery lead (Table 4). These emphasize automation, modularity, deployment consistency, and runtime visibility as the common backbone.

2) RQ2—CROSS-DOMAIN SEMANTIC ALIGNMENT

Stable alignments emerge from co-occurrence and taxonomy analysis: e.g., IaC → Declarative/Cloud-Native Setup, CI/CD → Orchestration/Modularization (see Table 6 categories; Figure 8 distribution). Evidence is stronger for Deployment Automation and Observability than for Security & Governance, as quantified in Table 8 and visualized in Figures 5 (empirical vs. conceptual) and 7 (evidence strength by category).

3) RQ3—EFFECTIVENESS OF RAG FOR BIDIRECTIONAL RECOMMENDATIONS

Against BM25 and Embed-only, RAG improves P@5 and nDCG@5 and attains RAE@5 = 1.27 (Table 5). Gains are largest in Security & Governance and Configuration & Environment Management, where semantic bridging counters sparse/heterogeneous terminology. Qualitative checks (Table 2) show concise, evidence-consistent justifications.

B. ANSWERS TO THE RESEARCH QUESTIONS

For researchers.

1. The normalized concept inventory and taxonomy (Tables 3–4; Table 6; Figure 8) form a reusable substrate for IR/NLP studies (retrievers, rerankers, ontologies).
2. The evaluation kit (Table 5, graded labels; κ reported in Section V-A) enables replicable benchmarking.
3. Evidence maps (Tables 7–9; Figures 5–7) make research gaps explicit—especially Security & Governance—and suggest where new empirical studies can add the most value.

For practitioners.

1. Use the taxonomy (Table 6; Figure 8) as a planning checklist:
 - Deployment Automation $\leftrightarrow$ Service choreography/modularity
 - Observability $\leftrightarrow$ Runtime diagnostics/telemetry
 - Configuration Mgmt $\leftrightarrow$ Declarative, stateless, cloud-native setups
2. In thinner-evidence areas (Security & Governance), run controlled pilots with explicit metrics (change-failure rate, policy-as-code coverage) before broad rollout.
3. Treat RAG outputs as decision aids: validate top suggestions against platform constraints, SLOs, compliance requirements, and keep a lightweight traceability log (mapping $\rightarrow$ change $\rightarrow$ outcome).

C. LLMS AND ARCHITECTURE-TO-OPERATIONS TRACEABILITY

RAG reduces the cognitive effort of tracing DevOps practice $\rightarrow$ architectural impact (and vice versa). In our setting, it (i) surfaces candidates consistent with the taxonomy, (ii) explains the rationale succinctly (Table 2), and (iii) maintains category coverage beyond high-frequency terms (Figure 6), yielding auditable recommendations rather than opaque generations.

D. LIMITATIONS AND VALIDITY CONSIDERATIONS (OVERVIEW; DETAILS IN SECTION VIII)

Findings depend on (i) the snapshot of sources, (ii) labeling subjectivity despite double annotation and $\kappa = 0.83$ (V-A), and (iii) possible domain bias (industry-heavy corpus). We discuss internal, external, construct, and conclusion validity and mitigations (ontology rules, adjudication, open data/code) in Section VII.

E. LESSONS LEARNED

1. **Normalization is critical.** Disambiguating Monitoring/Logging/Observability and Automation/IaC/Deployment Automation reduced cross-level errors and improved retrieval quality.
2. **Ablations matter.** Clear baselines (BM25, Embed-only) showed *where* LLM synthesis helps—particularly in sparse categories (Table 5; Figure 5 deltas by category).
3. **Explainability builds trust.** Short, evidence-grounded justifications (Table 2) increased annotator consistency and reviewer acceptance.
4. **Evidence gaps are actionable.** The imbalance in Security & Governance (Table 8; Figure 7) suggests a near-term agenda: policy-as-code, supply-chain controls (e.g., SLSA), zero-trust meshes with empirical evaluations.

F. CROSS-STUDY EMPIRICAL STRENGTH

Concrete instances underpin aggregate patterns. Table 9 lists representative studies with method and assessed support level; the strongest evidence clusters in case studies and migration analyses addressing IaC/CI-CD and cloud-native deployment. Weaker support often stems from security/traceability claims based on architectural reasoning rather than measured outcomes.

G. MULTI-INDUSTRY CASE STUDIES: SOFTWARE PLATFORM, BANKING, AND INSURANCE

We applied the framework in three settings:

1. **Software platform (mid-sized IT org; ~350 staff; ~1M users).**
Guided by RAG-recommended mappings, the team implemented targeted changes in deployment automation, observability, and configuration management. Reported outcomes over three months: shorter time-to-market, higher developer satisfaction, increased feature throughput, and reduced unit costs (Section V summary).
2. **Large-scale banking.** A tier-1 bank (millions of customers) used the taxonomy to align CI/CD + IaC with service-oriented release orchestration and runtime diagnostics. Results mirrored the first case with small deviations: slightly smaller relative gains in feature throughput (due to strict change-control windows) but larger gains in MTTR reduction attributable to upgraded telemetry and alert pipelines aligned to the Observability category (Figures 6–7 context).
3. **Large-scale insurance.** An insurance group adopted policy-as-code and artifact traceability under Security & Governance, plus declarative environment provisioning (Configuration). Improvements were comparable overall to the first case, with notably stronger reductions in change-failure rate (tight coupling between governance checks and deployment automation), but more gradual throughput gains due to regulatory sign-offs.

Across all three, we maintained traceability from RAG suggestion $\rightarrow$ implemented change $\rightarrow$ observed outcome. Differences across industries align with category emphasis: banking benefited most from Observability upgrades; insurance from Governance + Configuration controls; the software platform from Automation + Configuration. These variations are consistent with the evidence profiles in Tables 6–9 and Figures 5–8.

VII. THREATS TO VALIDITY

We discuss internal, external, construct, and conclusion validity, together with dataset/annotation limitations and reproducibility considerations. Throughout, we reference the empirical setup in Section V (Tables 2–9, Figures 5–8).

A. INTERNAL VALIDITY

1) POTENTIAL BIAS IN CONCEPT EXTRACTION AND MAPPING

Although mappings were manually curated and double-annotated, subjective interpretation of terms (e.g., *observability* vs. *logging*) can introduce bias. We mitigated this by (i) defining normalization rules and a fixed taxonomy (Table 6, Figure 8), (ii) using two independent annotators, and (iii) resolving disagreements via adjudication. Inter-rater agreement on a 30% stratified sample reached $\kappa = 0.83$ (V-A), indicating strong consistency. Residual risk remains for edge cases and cross-level terms despite normalization.

2) EVALUATION LEAKAGE AND PROMPT SENSITIVITY

RAG generation can be sensitive to retrieval pool composition and prompt phrasing. We controlled this with a fixed retriever index, frozen prompts, and ablations against BM25 and Embed-only (Table 5). Still, minor variance due to prompt phrasing and temperature cannot be completely eliminated.

B. EXTERNAL VALIDITY

1) GENERALIZABILITY ACROSS ORGANIZATIONS AND DOMAINS

The corpus comprises 84 peer-reviewed studies spanning 2016–2025 from major libraries, which helps diversity but excludes non-English/gray literature. Section VI-G reports deployments in software platform, banking, and insurance settings, suggesting applicability at multiple scales; however, regulated environments (e.g., finance/insurance) impose change-window and compliance constraints that may dampen or delay some gains. Results should therefore be generalized with attention to domain constraints and governance maturity.

2) CONCEPT DRIFT

DevOps/MSA terminology evolves (e.g., supply-chain security, platform engineering). The current index is a time-bounded snapshot; periodic re-indexing is needed to preserve external validity as practices change.

C. CONSTRUCT VALIDITY

1) ARE WE MEASURING WHAT WE CLAIM?

We operationalized *alignment* via (i) curated concept–concept mappings (Table 6), (ii) frequency evidence (Tables 3–4), and (iii) retrieval-quality metrics (Table 5). Ambiguities arise when concepts overlap or sit at different abstraction levels (e.g., automation vs. IaC vs. deployment automation). We mitigated this with ontology rules, canonical labels, and examples (Table 2). Residual risk: some semantically similar terms differ in practical implications, and our taxonomy—while explicit—still abstracts from full organizational context.

2) METRIC SUITABILITY

P@k and nDCG@k are standard IR metrics; RAE@k (RAG Alignment Efficiency) is a study-specific efficiency indicator. While it complements P@k/nDCG by relating retrieved

relevance to human-validated breadth, its interpretation depends on our labeling protocol; independent replication would strengthen its construct validity.

D. CONCLUSION VALIDITY

1) STATISTICAL STRENGTH AND INFERENCE LIMITS

The evaluation used 50 curated queries (10 per category). Results in Table 5 show consistent gains for RAG over baselines, but the sample size constrains statistical power and per-category subgroup analysis. Human relevance judgments (Table 2)—although double-annotated—remain partly subjective. Future replications with larger query sets and additional assessors (including industry experts) would further stabilize estimates.

2) ABLATION COVERAGE

We reported BM25 and Embed-only as baselines; stronger conclusions would benefit from adding rerankers or hybrid pipelines. That said, observed improvements are coherent with qualitative inspections and category-level deltas (Figures 5–7).

E. DATASET AND ANNOTATION LIMITATIONS

1) SELECTION BIAS AND COVERAGE

Inclusion criteria (English, peer-reviewed, 2016–2025) enhance quality but omit non-English and practice reports. Industry-heavy topics may be over-represented relative to niche academic threads.

2) ANNOTATION SUBJECTIVITY AND OVERLAP

Manual extraction can miss implicit concepts or misclassify overlapping terms. We applied normalization, exemplars, and adjudication; nonetheless, perfect semantic consistency is unattainable. Table 9 illustrates how evidence strength varies by method, reinforcing the need to separate validated from conceptual mappings (Tables 7–8).

3) STATIC SNAPSHOT

As new practices emerge (e.g., SBOM/SLSA, platform engineering patterns), the index must be refreshed; otherwise, recommendations may gradually misalign with state-of-practice.

F. REPRODUCIBILITY AND TRANSPARENCY

We improve reproducibility by (i) fixing retrieval indices and prompts for the 50-query run, (ii) publishing taxonomy definitions and labeling rubrics (referenced in V-A/V-E), and (iii) reporting κ , per-category metrics, and examples (Tables 2, 5; Figures 5–7). We encourage third-party replication with expanded query pools, additional annotators, and domain-specific corpora (e.g., finance/insurance) to test portability.

1) SUMMARY

Mitigations (double-annotation with $\kappa = 0.83$, ontology/normalization, fixed baselines, and transparent metrics)

reduce—but do not eliminate—threats across validity dimensions. The multi-industry deployments support external validity claims qualitatively, while Section V's quantitative results establish performance improvements. Ongoing dataset refresh, broader evaluators, and extended baselines are the most direct ways to further strengthen validity.

VIII. CONCLUSION AND FUTURE WORK

This paper presented an empirical and AI-supported approach for aligning DevOps practices with microservice architecture (MSA) principles. From a curated corpus of 84 peer-reviewed sources (2016–2025), we extracted and normalized concepts, constructed more than 200 DevOps↔MSA mappings, and organized them into a five-category taxonomy: Deployment Automation & Orchestration, Observability & Runtime Diagnostics, Scalability & Modularity, Security & Governance, and Configuration & Environment Management. We operationalized these insights through a retrieval-augmented generation (RAG) framework that couples sentence-transformer retrieval with LLM reasoning to recommend bidirectional concept mappings with concise, evidence-grounded justifications.

Quantitatively, RAG outperformed BM25 and Embed-only across standard IR metrics (Table 5), and qualitatively its explanations aligned with human assessments (Table 2). The empirical synthesis (Tables 7–9; Figures 5–7) shows that DeploymentAutomation and Observability are the most strongly validated areas, while Security & Governance remains comparatively under-validated, indicating a clear research opportunity. Multiple industrial deployments—including a software platform, a tier-1 bank, and a national insurance group—suggest the framework's practical utility at different scales and regulatory contexts, with variations that reflect domain emphasis (Section VI-G).

Overall, the study contributes: (i) a reusable, normalized concept dataset and taxonomy; (ii) a RAG-based decision aid that improves architecture-to-operations traceability; and (iii) an empirical map that highlights where evidence is strong versus emerging. Taken together, these artifacts help researchers structure comparative studies and help practitioners plan, justify, and trace DevOps/MSA decisions.

A. FUTURE WORK

1. **Broader and fresher evidence.** Expand the corpus to non-English and gray literature; add sector-specific corpora (e.g., finance, telecom, public sector) and periodically refresh the index to address concept drift (e.g., SBOM/SLSA, platform engineering, policy-as-code).
2. **Stronger evaluations.** Enlarge the query set, include additional annotators (industry experts), and add hybrid/reranking baselines and task-level evaluations (e.g., downstream change-failure rate/MTTR impacts).
3. **Methodological enhancements.** Incorporate uncertainty-aware retrieval, prompt-robust generation, and explanation auditing to further strengthen construct and conclusion validity.

4. **Toolchain integration.** Package the framework as an open, extensible toolkit that plugs into CI/CD platforms and architecture workbenches to provide in-flow recommendations, traceability links (mapping → change → outcome), and governance reports.
5. **Education and training.** Use the taxonomy and dataset to design curricula and interactive labs that connect DevOps practices to microservice design concerns with explainable, evidence-based exercises.

By releasing the dataset, taxonomy, and implementation artifacts, we aim to catalyze replication and extension. We see empirically grounded RAG as a practical path toward evidence-informed, auditable decisions that bridge architecture and operations in complex software systems.

STATEMENTS AND DECLARATIONS

The authors have no competing interests to declare that are relevant to the content of this article.

DATA AVAILABILITY

All artifacts used in this study are publicly available. The working repository is hosted on GitHub, and an immutable, citable snapshot of the exact version used in our analyses is archived on Zenodo. Persistent links and the commit/DOI identifiers are provided in Section IV-A.

REFERENCES

- [1] A. Sen and I. Skrobot, "Implementation of DevOps paradigm to deployment and provisioning of microservices," *Issues Inf. Syst.*, vol. 22, no. 1, pp. 136–148, 2021, doi: [10.48009/1_iis_2021_136-148](#).
- [2] X. Zhou, J. Xu, X. Li, L. Cao, L. Li, Y. Wang, S. Li, and H. Liu, "A blockchain-based and microservices-architected software composition analysis system," *J. Softw., Evol. Process*, vol. 36, no. 10, Oct. 2024, Art. no. e2675, doi: [10.1002/smr.2675](#).
- [3] B. L. Berg, "Service-oriented architecture over UDSO-CAN," in *Proc. IEEE 100th Veh. Technol. Conf. (VTC-Fall)*, Washington, DC, USA, Oct. 2024, pp. 1–7, doi: [10.1109/vtc2024-fall63153.2024.10757791](#).
- [4] S. K. Rai, "Demystifying cloud native data engineering architectures," *Int. J. Inf. Technol. Manage. Inf. Syst.*, vol. 16, no. 2, pp. 959–993, Mar. 2025, doi: [10.34218/ijitmis_16_02_062](#).
- [5] E. M. I. M. Ekanayaka, J. K. K. H. Thathsarani, D. S. Karunanayaka, N. Kuruwitaarachchi, and N. Skandhakumar, "Enhancing devops infrastructure for efficient management of microservice applications," in *Proc. IEEE Int. Conf. e-Business Eng. (ICEBE)*, Nov. 2023, pp. 63–68, doi: [10.1109/ICEBE59045.2023.00035](#).
- [6] X. Zhou, H. Huang, H. Zhang, X. Huang, D. Shao, and C. Zhong, "A cross-company ethnographic study on software teams for DevOps and microservices: Organization, benefits, and issues," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng., Softw. Eng. Pract. (ICSE-SEIP)*, Pittsburgh, PA, USA, May 2022, pp. 1–10, doi: [10.1145/3510457.3513054](#).
- [7] C. Reile, M. Chadha, V. Hauner, A. Jindal, B. Hofmann, and M. Gerndt, "Bunk8s: Enabling easy integration testing of microservices in Kubernetes," in *Proc. IEEE Int. Conf. Softw. Anal., Evol. Reengineering (SANER)*, Mar. 2022, pp. 459–463, doi: [10.1109/SANER53432.2022.00062](#).
- [8] F. Rademacher, S. Sachweh, and A. Zündorf, "Deriving microservice code from underspecified domain models using DevOps-enabled modeling languages and model transformations," in *Proc. 46th Euromicro Conf. Softw. Eng. Adv. Appl. (SEAA)*, Aug. 2020, pp. 229–236, doi: [10.1109/SEAA51224.2020.00047](#).
- [9] F. Rademacher, J. Sorgalla, S. Sachweh, and A. Zündorf, "Viewpoint-specific model-driven microservice development with interlinked modeling languages," in *Proc. IEEE Int. Conf. Service-Oriented Syst. Eng. (SOSE)*, Apr. 2019, pp. 57–66, doi: [10.1109/SOSE.2019.00018](#).

- [10] A. W. Hou, K.-H. Hsu, and Y.-Y. Chen, "Addressing security issues during decomposing golang monolithic applications into microservices through code analysis," in *Proc. 10th Int. Conf. Appl. Syst. Innov. (ICASI)*, Apr. 2024, pp. 445–447, doi: [10.1109/icasi60819.2024.10547752](https://doi.org/10.1109/icasi60819.2024.10547752).
- [11] U. Vora, "Identifying and architecting microservices for edge computing," in *Proc. IEEE 22nd Int. Conf. Softw. Archit. Companion (ICSA-C)*, Mar. 2025, pp. 173–174, doi: [10.1109/ICSA-C65153.2025.00032](https://doi.org/10.1109/ICSA-C65153.2025.00032).
- [12] T. Zhong, Y. Teng, S. Ma, J. Chen, and S. Yu, "A microservices identification method based on spectral clustering for industrial legacy systems," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, Dec. 2023, pp. 1331–1337, doi: [10.1109/GCWkshps58843.2023.10464508](https://doi.org/10.1109/GCWkshps58843.2023.10464508).
- [13] Y.-W. Huang, S.-P. Ma, and S. K. Wang, "MsdoBot: An extensible chatbot platform for microservice development and operations," in *Proc. IEEE Int. Conf. e-Business Eng. (ICEBE)*, Oct. 2022, pp. 124–129, doi: [10.1109/ICEBE55470.2022.00030](https://doi.org/10.1109/ICEBE55470.2022.00030).
- [14] A. Shantyr, "Methodology for quantitative assessment of quality characteristics in distributed systems and microservices," in *Proc. IEEE 5th KhPI Week Adv. Technol. (KhPIWeek)*, Oct. 2024, pp. 1–6, doi: [10.1109/KHPI-Week61434.2024.10878097](https://doi.org/10.1109/KHPI-Week61434.2024.10878097).
- [15] B. Mahmoudi, I. Trabelsi, D. Tamzalit, N. Moha, and Y. Guéhéneuc, "BOAM: A business oriented identification approach of microservices within legacy systems," in *Proc. Service-Oriented Comput. (ICSOC)*, 2024, pp. 123–137, doi: [10.1007/978-981-96-0808-9_10](https://doi.org/10.1007/978-981-96-0808-9_10).
- [16] J. Y. Zhang and Y. Zhang, "Quantitative DevSecOps metrics for cloud-based web microservices," *IEEE Access*, vol. 12, pp. 160317–160334, 2024, doi: [10.1109/ACCESS.2024.3486314](https://doi.org/10.1109/ACCESS.2024.3486314).
- [17] M. Kim, "Guidelines for designing an IT undergraduate course on cloud-native technologies for microservice architectures," *Int. J. Internet, Broadcast. Commun.*, vol. 17, no. 1, pp. 352–364, Feb. 2025, doi: [10.7236/IJIBC.2025.17.1.352](https://doi.org/10.7236/IJIBC.2025.17.1.352).
- [18] S. Moreschini, S. M. Pour, I. Lanese, D. Balouek-Thomert, J. Bogner, X. Li, F. Pecorelli, J. Soldani, E. Truyen, and D. Taibi, "AI techniques in the microservices life-cycle: A systematic mapping study," *Comput.*, vol. 107, no. 4, 2025, doi: [10.1007/s00607-025-01432-z](https://doi.org/10.1007/s00607-025-01432-z).
- [19] H. Rodrigues, A. Rito Silva, and A. Avritzer, "Assessment of performance and its scalability in microservice architectures: Systematic literature review," *J. Syst. Softw.*, vol. 230, Dec. 2025, Art. no. 112500, doi: [10.1016/j.jss.2025.112500](https://doi.org/10.1016/j.jss.2025.112500).
- [20] Y. Hammad, A. Al-Said Ahmad, and P. Andras, "An empirical study on the performance overhead of code instrumentation in containerised microservices," *J. Syst. Softw.*, vol. 230, Dec. 2025, Art. no. 112573, doi: [10.1016/j.jss.2025.112573](https://doi.org/10.1016/j.jss.2025.112573).
- [21] N. Kaushik, H. Kumar, and V. Raj, "A systematic review of QoS enhancement techniques in microservices architecture," *Comput. Electr. Eng.*, vol. 127, Oct. 2025, Art. no. 110550, doi: [10.1016/j.compeleceng.2025.110550](https://doi.org/10.1016/j.compeleceng.2025.110550).
- [22] P. Dakić, "Software compliance in various industries using CI/CD, dynamic microservices, and containers," *Open Comput. Sci.*, vol. 14, no. 1, Jul. 2024, doi: [10.1515/comp-2024-0013](https://doi.org/10.1515/comp-2024-0013).
- [23] J. McZara, S. Kafle, and D. Shin, "Modeling and analysis of dependencies between microservices in DevSecOps," in *Proc. IEEE Int. Conf. Smart Cloud (SmartCloud)*, Washington, DC, USA, Nov. 2020, pp. 140–147, doi: [10.1109/SmartCloud49737.2020.00034](https://doi.org/10.1109/SmartCloud49737.2020.00034).
- [24] M. S. Floroiu, S. Russo, L. Giamattei, A. Guerriero, I. Malavolta, and R. Pietrantuono, "Anomaly detection and root cause analysis of microservices energy consumption," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, Shenzhen, China, Jul. 2024, pp. 590–600, doi: [10.1109/ICWS62655.2024.00079](https://doi.org/10.1109/ICWS62655.2024.00079).
- [25] G. Budakoglu and H. Emekci, "Unveiling the power of large language models: A comparative study of retrieval-augmented generation, fine-tuning, and their synergistic fusion for enhanced performance," *IEEE Access*, vol. 13, pp. 30936–30951, 2025, doi: [10.1109/ACCESS.2025.3542334](https://doi.org/10.1109/ACCESS.2025.3542334).
- [26] I. Trabelsi, B. Popa, J. Pereyrol, P.-O. Beaulieu, and N. Moha, "Micro-Matic: Fully automated microservices identification approach from monolithic systems," in *Proc. ACM/IEEE 6th Int. Workshop Softw. Eng. Res. Practices Internet Things*, Lisbon, Portugal, Apr. 20, 2024, pp. 7–13, doi: [10.1145/3643794.3648283](https://doi.org/10.1145/3643794.3648283).
- [27] G. Fuentes-Quijada, F. Ruiz-González, and A. Caro, "Enterprise architecture and IT governance to support the BizDevOps approach: A systematic mapping study," *Inf. Syst. Frontiers*, vol. 27, no. 3, pp. 865–888, Jun. 2025, doi: [10.1007/s10796-024-10473-2](https://doi.org/10.1007/s10796-024-10473-2).
- [28] D. Trihinas, A. Tryfonos, M. D. Dikaiakos, and G. Pallis, "DevOps as a service: Pushing the boundaries of microservice adoption," *IEEE Internet Comput.*, vol. 22, no. 3, pp. 65–71, May 2018, doi: [10.1109/MIC.2018.032501519](https://doi.org/10.1109/MIC.2018.032501519).
- [29] G. Morais, "Enabling systematic microservices reuse through DevOps," in *Proc. CAiSE Doctoral Consortium*, vol. 3407, 2023, pp. 1–10. [Online]. Available: <https://ceur-ws.org/Vol-3407/paper1.pdf>
- [30] D. T. Vu, "CI/CD automation's impact on microservices project management," Bachelor's thesis, Turku Univ. Appl. Sci., Turku, Finland, 2024. [Online]. Available: <https://www.theseus.fi/handle/10024/876023>
- [31] Z. Haider and J. Yang, "Revolutionizing enterprise architecture: Harnessing AI and cloud synergy with DevOps integration," *ResearchGate*, Nov. 2024, doi: [10.13140/RG.2.2.33117.63200](https://doi.org/10.13140/RG.2.2.33117.63200).
- [32] S. Kothapalli, M. Nizamuddin, R. R. Talla, and J. C. S. Gummadi, "DevOps and software architecture: Bridging the gap between development and operations," *J. Comput. Digit. Technol.*, vol. 2, no. 1, pp. 51–64, 2024. [Online]. Available: https://www.americandigits.com/?/2024_devops_and_software_architecture_bridging_the_gap_between_development_and_operations.pdf
- [33] V. Kunchenapalli, "A case study: How DevOps enabled easy monolithic to microservice migration," *Int. J. Comput. Eng. Technol. (IJCET)*, vol. 15, no. 5, pp. 105–116, Sep. 2024, doi: [10.5281/zenodo.13737601](https://doi.org/10.5281/zenodo.13737601).
- [34] H. Michael Ayas, R. Hebig, and P. Leitner, "An empirical investigation on the competences and roles of practitioners in microservices-based architectures," *J. Syst. Softw.*, vol. 213, Jul. 2024, Art. no. 112055, doi: [10.1016/j.jss.2024.112055](https://doi.org/10.1016/j.jss.2024.112055).
- [35] S.-K. Wang, S.-P. Ma, G.-H. Lai, and C.-H. Chao, "ChatOps for microservice systems: A low-code approach using service composition and large language models," *Future Gener. Comput. Syst.*, vol. 161, pp. 518–530, Dec. 2024, doi: [10.1016/j.future.2024.07.029](https://doi.org/10.1016/j.future.2024.07.029).
- [36] R. F. P. Heikel, M. B. Guayuan, M. L. Cremona, H. Nemeth, and J. C. Mello-Román, "Development of applications based on microservices-case study of ministry of economy and finance of Paraguay," in *Proc. L. Latin Amer. Comput. Conf. (CLEI)*, Paraguay, Aug. 2024, pp. 1–8, doi: [10.1109/clei64178.2024.10700123](https://doi.org/10.1109/clei64178.2024.10700123).
- [37] L. Giamattei, A. Guerriero, R. Pietrantuono, S. Russo, I. Malavolta, T. Islam, M. Dinga, A. Koziolok, S. Singh, M. Armbruster, J. M. Gutierrez-Martinez, S. Caro-Alvaro, D. Rodriguez, S. Weber, J. Henss, E. F. Vogelien, and F. S. Panojo, "Monitoring tools for DevOps and microservices: A systematic grey literature review," *J. Syst. Softw.*, vol. 208, Feb. 2024, Art. no. 111906, doi: [10.1016/j.jss.2023.111906](https://doi.org/10.1016/j.jss.2023.111906).
- [38] C. A. Supit, A. Abdullah Pangeran, K. O. C. Laban, I. Irena Gutandjaja, and A. Ramadhan, "Incorporating cloud native architecture and DevOps culture to improve company agility," in *Proc. 3rd Int. Conf. Intell. Cybern. Technol. Appl. (ICICyTA)*, Dec. 2023, pp. 290–294. [Online]. Available: https://www.researchgate.net/?/Incorporating_Cloud_Native_Architecture_and_DevOps_Culture_to_Improve_Company_Agility
- [39] D. Badampudi, M. Usman, and X. Chen, "Large scale reuse of microservices using CI/CD and innersource practices—A case study," *Empirical Softw. Eng.*, vol. 30, no. 2, p. 41, Mar. 2025, doi: [10.1007/s10664-024-10595-w](https://doi.org/10.1007/s10664-024-10595-w).
- [40] B. M. Harve, D. M. Bidkar, M. S. Krishnappa, G. Pandey, V. Jayaram, P. K. Veerapaneni, and G. Mehta, "The cloud-native revolution: Microservices in a cloud-driven world," in *Proc. Int. Conf. Intell. Cybern. Technol. Appl. (ICICyTA)*, Dec. 2024, pp. 1–9, doi: [10.1109/ICICyTA64807.2024.10913359](https://doi.org/10.1109/ICICyTA64807.2024.10913359).
- [41] J. Cui, "Comparative analysis of DevOps, DevSecOps, and AIOps in software development architectures: Impacts on software quality and development efficiency," *Tech. Rep.*, Nov. 2024, doi: [10.13140/RG.2.2.32206.68162](https://doi.org/10.13140/RG.2.2.32206.68162).
- [42] R. Rodrigues-Filho, I. Sene, B. Porter, L. F. Bittencourt, F. Kon, and F. M. Costa, "Exploring emergent microservice evolution in elastic deployment environments," *J. Syst. Softw.*, vol. 219, Jan. 2025, Art. no. 112252, doi: [10.1016/j.jss.2024.112252](https://doi.org/10.1016/j.jss.2024.112252).
- [43] A. K. Akula, "Leveraging AWS and Java microservices: An analysis of Amazon's scalable e-commerce architecture," *Int. J. for Res. Appl. Sci. Eng. Technol.*, vol. 12, no. 9, pp. 1051–1063, Sep. 2024, doi: [10.22214/ijraset.2024.64275](https://doi.org/10.22214/ijraset.2024.64275).

- [44] J. Soldani, R. Amadini, A. Brogi, S. Forti, S. Giallorenzo, P. Plebani, M. Vitali, and G. Zavattaro, "Towards sustainable deployment of microservices over the cloud-IoT continuum, with FREEDA," in *Proc. 4th Workshop Flexible Resource Appl. Manage. Edge*, Jun. 2024, pp. 1–4, doi: [10.1145/3659994.3660311](https://doi.org/10.1145/3659994.3660311).
- [45] B. Venkata, "End-to-end CI/CD deployment of RESTful microservices in the cloud," *Int. J. Core Eng. Manage.*, vol. 6, no. 8, pp. 262–268, Aug. 2020, doi: [10.5281/zenodo.15020514](https://doi.org/10.5281/zenodo.15020514).
- [46] A. A. Azeta, D. A. Iboroma, I. O. Oyeyemi, F. O. Abimbola, and B. Ogunde, "A DevOps software architecture for recommender systems in digital library," in *Proc. Int. Conf. eLearning*, 2023.
- [47] K. Irfan and M. Daniel, "AI-augmented DevOps: A new paradigm in enterprise architecture and cloud management," *Tech. Rep.*, 2024, doi: [10.13140/RG.2.2.24729.02406](https://doi.org/10.13140/RG.2.2.24729.02406).
- [48] H. Saeed and M. Daniel, "Smart enterprise architecture: Leveraging AI, cloud, and agile DevOps practices," *Tech. Rep.*, 2024, doi: [10.13140/RG.2.2.18018.13761](https://doi.org/10.13140/RG.2.2.18018.13761).
- [49] D. R. Martinez and B. M. Kifle, *Artificial Intelligence: A Systems Approach From Architecture Principles To Deployment*. Cambridge, MA, USA: MIT Press, 2024, doi: [10.7551/mitpress/14806.001.0001](https://doi.org/10.7551/mitpress/14806.001.0001).
- [50] S. Arya, D. Chauhan, Tanishq, S. Anand, and O. Sharma, "Beyond monoliths: An in-depth analysis of microservices adoption in the era of Kubernetes," in *Proc. 1st Int. Conf. Adv. Comput. Emerg. Technol. (ACET)*, Aug. 2024, pp. 1–6. [Online]. Available: https://www.researchgate.net/publication/390172110_Beyond_Monoliths_?
- [51] S. J. Warnett and U. Zdun, "On the understandability of MLOps system architectures," *IEEE Trans. Softw. Eng.*, vol. 50, no. 5, pp. 1015–1039, May 2024, doi: [10.1109/TSE.2024.3367488](https://doi.org/10.1109/TSE.2024.3367488).
- [52] M. Amrit, M. P. Akhil, M. Rakshith, and B. S. Shylaja, "Microservices evolving DevOps pipelines," *Int. Res. J. Modernization Eng. Technol. Sci. (IRJMETS)*, vol. 4, no. 9, pp. 632–639, Sep. 2022, doi: [10.56726/irjrmets29808](https://doi.org/10.56726/irjrmets29808).
- [53] A. P. Sundareswaran, P. (A. S. Kushwaha, and S. University, "Microservices and DevOps: Accelerating eCommerce delivery," *Int. J. Res. Humanities Social Sci.*, vol. 13, no. 3, pp. 112–132, Mar. 2025, doi: [10.63345/ijrhrs.net.v13.i3.7](https://doi.org/10.63345/ijrhrs.net.v13.i3.7).
- [54] Q. Cooper, "Expedited load tests for CI/CD microservice applications," Master's thesis, Univ. Calgary, Calgary, AB, Canada, 2024. [Online]. Available: <https://prism.ucalgary.ca/download>
- [55] P. Singh Yadav, "Migration to serverless architectures: A pathway to efficient cloud computing," *Int. J. Sci. Res. (IJSR)*, vol. 13, no. 10, pp. 275–283, Oct. 2024, doi: [10.21275/sr241002075058](https://doi.org/10.21275/sr241002075058).
- [56] R. Vogl and M. Blank-Burian, "An update on the Münster university cloud-technical architecture and user adoption," in *Proc. EPIc Ser. Comput.*, vol. 105, 2025, pp. 126–135, doi: [10.29007/dd24](https://doi.org/10.29007/dd24).
- [57] D. Kohli, J. Kumar, J. Saini, S. Aggarwal, Y. Singh, and A. Garg, "Implementing microservice architecture in e-commerce with DevOps practice," in *Proc. Int. Conf. Intell. Syst. Cybersecurity (ISCS)*, Gurugram, India, May 2024, pp. 1–6, doi: [10.1109/iscs61804.2024.10581082](https://doi.org/10.1109/iscs61804.2024.10581082).
- [58] F. Ponce, S. Forti, J. Soldani, and A. Brogi, "Enhancing failure resilience of cloud-edge microservices: The FREEDA approach," in *Proc. Service-Oriented Cloud Comput. (ESOCC)*, 2025, pp. 105–111, doi: [10.1007/978-3-031-84617-5_9](https://doi.org/10.1007/978-3-031-84617-5_9).
- [59] I. Koren, F. Rinker, K. Meixner, M. Kröger, and M. Zeng, "Implementing DevOps practices in CPPS using microservices and GitOps," in *Proc. IEEE 28th Int. Conf. Emerg. Technol. Factory Autom. (ETFA)*, Sep. 2023, pp. 1–4, doi: [10.1109/ETFA54631.2023.10275433](https://doi.org/10.1109/ETFA54631.2023.10275433).
- [60] P. Bělka, "Cloud-native and microservice application development," in *Proc. 20th Int. Conf. Emerg. eLearning Technol. Appl. (ICETA)*, 2022, pp. 239–244.
- [61] A. Johansson and S. Paulsson, "Microservice integration testing with hardware-in-the-loop in CI/CD pipelines," Master's thesis, Chalmers Univ. Technol., Gothenburg, Sweden, 2024. [Online]. Available: <http://hdl.handle.net/20.500.12380/308696>
- [62] A. AbouZaid, P. J. Barclay, C. Chrysoulas, and N. Pitropakis, "Building a modern data platform based on the data lakehouse architecture and cloud-native ecosystem," *Discover Appl. Sci.*, vol. 7, no. 3, Feb. 2025, doi: [10.1007/s42452-025-06545-w](https://doi.org/10.1007/s42452-025-06545-w).
- [63] B. P. Pendyala, "The convergence of platform engineering and service automation: Transforming enterprise cloud architecture," *Int. J. Inf. Technol. Manage. Inf. Syst.*, vol. 16, no. 2, pp. 225–239, Mar. 2025, doi: [10.34218/ijtmis_16_02_016](https://doi.org/10.34218/ijtmis_16_02_016).
- [64] B. Sun, H. Zhou, G. Song, and Q. Zhang, "Research on the technology architecture of full-stack cloud-native hosted cloud," in *Proc. 11th Int. Conf. Mach. Intell. Theory Appl. (MiTA)*, Jul. 2024, pp. 1–7, doi: [10.1109/mita60795.2024.10751724](https://doi.org/10.1109/mita60795.2024.10751724).
- [65] M. Matt and W. Jeremiah. (2023). *Modern Enterprise Architecture in the AI Era: Integrating Cloud, DevOps, and DataOps*. Accessed: Nov. 1, 2025. [Online]. Available: https://www.researchgate.net/publication/390172110_
- [66] J. Flora and N. Antunes, "Doing more with less? A study on models for intrusion detection in microservices," in *Proc. 19th Eur. Dependable Comput. Conf. (EDCC)*, Leuven, Belgium, Apr. 2024, pp. 49–56, doi: [10.1109/edcc61798.2024.00021](https://doi.org/10.1109/edcc61798.2024.00021).
- [67] A. Delgado, F. García, and F. Ruíz, "BizDevOps support for business Process microservices-based applications," in *Proc. ICSOC Workshops, (ASOCA)*, Nov. 2023, pp. 274–286, doi: [10.1007/978-3-031-26507-5_22](https://doi.org/10.1007/978-3-031-26507-5_22).
- [68] M. Waseem, P. Liang, and M. Shahin, "A systematic mapping study on microservices architecture in DevOps," *J. Syst. Softw.*, vol. 170, Dec. 2020, Art. no. 110798, doi: [10.1016/j.jss.2020.110798](https://doi.org/10.1016/j.jss.2020.110798).
- [69] O. R. Kuma and A. Narang, "Securing microservices: Challenges and solutions," *Int. J. Innov. Res. Comput. Sci. Technol.*, vol. 13, no. 1, pp. 58–61, Jan. 2025. [Online]. Available: <https://ijrscst.org/DOC/8-Securing-Microservices-Challenges-and-Solutions.pdf>
- [70] M. Matthew, J. Anderson, and V. Ogunrinde. (2022). *AI and Cloud Computing Synergy: Transforming Enterprise Architecture With DevOps and DataOps*. Accessed: Nov. 1, 2025. [Online]. Available: https://www.researchgate.net/publication/386254108_
- [71] K. Gangu and R. Mishra, "DevOps and continuous delivery in cloud-based CDN architectures," *Int. J. Res. All Subj. Multi Lang. (IJRSMML)*, vol. 13, no. 1, pp. 69–90, Jan. 2025. [Online]. Available: <https://ijrsmml.org/wp-content/uploads/2025/01/?pdf>
- [72] A. Balalaie, A. Heydarnoori, and P. Jamshidi, "Microservices architecture enables DevOps: Migration to a cloud-native architecture," *IEEE Softw.*, vol. 33, no. 3, pp. 42–52, May 2016, doi: [10.1109/MS.2016.64](https://doi.org/10.1109/MS.2016.64).
- [73] K. Nadeem and S. Aslam, "Cloud-native DevOps strategies: Redefining enterprise architecture with artificial intelligence," *Tech. Rep.*, Nov. 2024, doi: [10.13140/RG.2.2.13823.83369](https://doi.org/10.13140/RG.2.2.13823.83369).
- [74] R. R. Vangala, "Adaptive resilience framework: A comprehensive study on dynamic orchestration and auto-scaling of microservices in cloud-native systems," *Int. J. Comput. Eng. Technol. (IJCET)*, vol. 9, no. 6, pp. 278–288, 2018. [Online]. Available: https://iaeme.com/IJ/IJCET_09_06_029.pdf
- [75] J.-P. Busch, L. Reiher, and L. Eckstein, "Enabling the deployment of any-scale robotic applications in microservice architectures through automated containerization," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Yokohama, Japan, May 2024, pp. 17650–17656, doi: [10.1109/ICRA57147.2024.10611586](https://doi.org/10.1109/ICRA57147.2024.10611586).
- [76] A. R. Bommarreddy, "Automating DevOps in insurance: A case study of CI/CD implementation with micro-frontend architecture," *Int. J. Res. Comput. Appl. Inf. Technol.*, vol. 7, no. 2, pp. 2330–2339, Dec. 2024, doi: [10.34218/ijrcat_07_02_171](https://doi.org/10.34218/ijrcat_07_02_171).
- [77] L. Patan, "Leveraging cloud-native architecture for scalable and resilient enterprise applications: A comprehensive analysis," *Int. J. Comput. Eng. Technol. (IJCET)*, vol. 15, no. 5, pp. 583–591, Sep. 2024, doi: [10.5281/zenodo.13861921](https://doi.org/10.5281/zenodo.13861921).
- [78] R. Eramo, B. Said, M. Oriol, H. Bruneliere, and S. Morales, "An architecture for model-based and intelligent automation in DevOps," *J. Syst. Softw.*, vol. 217, Nov. 2024, Art. no. 112180, doi: [10.1016/j.jss.2024.112180](https://doi.org/10.1016/j.jss.2024.112180).
- [79] U. Tiwari, S. Rai, I. Gaur, and S. Khurana, "Dynamic container optimization for efficient microservices deployment," in *Proc. Int. Conf. Electr. Electron. Comput. Technol. (ICEECT)*, Aug. 2024, pp. 1–6, doi: [10.1109/iceect61758.2024.10739185](https://doi.org/10.1109/iceect61758.2024.10739185).
- [80] K. Oha, "Advancements in microservice architectures: Tackling data communication, scalability, and CI/CD automation challenges," Bachelor's thesis, Waal Univ. Appl. Sci., Kamp-Lintfort, Germany, Aug. 6, 2024. [Online]. Available: <https://opus4.kobv.de/opus4-rhein-waal/files/2046/KO-ADVANCEMENTS-IN-MICROSERVICE-ARCHITECTURES.pdf>

- [81] H. T. Nguyen, M. Usman, and R. Buyya, "QFaaS: A serverless function-as-a-service framework for quantum computing," *Future Gener. Comput. Syst.*, vol. 154, pp. 281–300, May 2024, doi: [10.1016/j.future.2024.01.018](https://doi.org/10.1016/j.future.2024.01.018).
- [82] A. TaeiZadeh, Z. Lotfi, and A. J. Ramadhan, "Microservices boundary determination migration in DevOps: A case study," *BIO Web Conf.*, vol. 97, p. 00122, Apr. 2024, doi: [10.1051/bioconf/20249700122](https://doi.org/10.1051/bioconf/20249700122).
- [83] U. Faseeha, H. Jamil Syed, F. Samad, S. Zehra, and H. Ahmed, "Observability in microservices: An in-depth exploration of frameworks, challenges, and deployment paradigms," *IEEE Access*, vol. 13, pp. 72011–72039, 2025, doi: [10.1109/ACCESS.2025.3562125](https://doi.org/10.1109/ACCESS.2025.3562125).
- [84] M. Abdulrahman, R. Alqahtani, and A. Bashar, "Review of opportunities, challenges and solutions for FaaS performance and security management," in *Proc. Int. Conf. Inventive Comput. Technol. (ICICT)*, Lalitpur, Nepal, Apr. 2024, pp. 1392–1398, doi: [10.1109/icict60155.2024.10544765](https://doi.org/10.1109/icict60155.2024.10544765).



ALI MOVAGHAR (Senior Member, IEEE) received the B.S. degree in electrical engineering from the University of Tehran, in 1977, and the M.S. and Ph.D. degrees in computer, information, and control engineering from the University of Michigan, in 1979 and 1985, respectively. He is currently an Emeritus Professor with the Department of Computer Engineering, Sharif University of Technology, Tehran, Iran, where he has been a Faculty Member, since 1993.

...



EHSAN AZIZI KHADEM received the M.S. degree in computer engineering from Tarbiat Modares University, Tehran, Iran, in 2005. He is currently pursuing the Ph.D. degree in computer engineering with the CE Department, Sharif University of Technology, Tehran.